第五週筆記

主題	集成學習、Bagging、隨機森林、Adaboost
日期	October 7, 2026 → October 13, 2026
相關課程	🧠 機器學習與深度學習
課堂簡報	第五週簡報

本週主題：經典機器學習的分類與迴歸（二）（Classical Machine Learning: Classification and Regression II）

> 💡 **這堂課的主線：一個模型可能不夠穩，怎麼讓多個模型一起預測？**
> 平行集成（Parallel Ensemble）：各自學習，最後整合，主要減少預測的變異。
> 序列集成（Sequential Ensemble）：一輪接一輪，後面專門補前面做不好的地方。

1. 開場：從聊天機器人到持續型智能體
2. 支持向量機（Support Vector Machine, SVM）
    2.1 它要解決什麼問題？
    2.2 支持向量（Support Vectors）是哪些點？
    2.3 資料不能用直線分開怎麼辦？
3. 怎麼比較不同分類模型？
    3.1 交叉驗證（Cross-validation, CV）
    3.2 不只看平均分數
    3.3 課堂示範結果與限制
4. 集成學習（Ensemble Learning）為什麼有用？
    4.1 多個模型一起決定

- 4.2 關鍵是多樣性（Diversity）
- 4.3 老師的兩個條件
- 4.4 集成失誤率怎麼算？
- 4.5 投票分類器（Voting Classifier）

5. 為什麼 Bagging 常用不穩定的基礎模型？
6. 自助聚合（Bagging）
   - 6.1 自助抽樣（Bootstrap Sampling）
   - 6.2 Bagging 的流程
   - 6.3 課堂的一維小例子（Toy Example）
   - 6.4 為什麼只看到約 63.2% 的不同資料？
   - 6.5 袋外資料（Out-of-bag, OOB）
   - 6.6 老師提到的實作參數
7. 隨機森林（Random Forest, RF）
   - 7.1 為什麼 Bagging 還不夠？
   - 7.2 在每個節點，只開放一部分特徵
   - 7.3 手寫數字辨識示範
8. 特徵重要性與評估報告
   - 8.1 特徵重要性（Feature Importance）
   - 8.2 老師的校舍耐震案例
   - 8.3 SHAP 是什麼？
   - 8.4 評估報告的英文術語
9. 極端隨機樹（Extremely Randomized Trees, Extra Trees, ET）
   - 9.1 老師比較了哪些變因？
   - 9.2 最重要的是測試表現
10. 從平行集成走向序列集成（Sequential Ensemble）
11. 自適應提升（Adaptive Boosting, AdaBoost）
    - 11.1 核心：上次做錯的題目，下次多注意
    - 11.2 先分清兩種權重

11.3 必要公式：二元 AdaBoost
11.4 老師的 10 個點例子
12. 本堂最後預告：梯度提升與 XGBoost
12.1 梯度提升（Gradient Boosting）
12.2 極端梯度提升（eXtreme Gradient Boosting, XGBoost）
13. 複習時，先能回答這 8 題
課堂來源與查核補充

---

1. 開場：從聊天機器人到持續型智能體

對照逐字稿：約 00:00–00:05。

老師用 AI 產品演進帶出一件事：學技術之外，也要思考它怎麼用在自己的專業。

- 聊天機器人（Chatbot）：問一句、答一句。
- 推理（Reasoning）與工具使用（Tool Use）：先思考，再查資料或呼叫工具。
- 智能體（Agent）：能規劃並執行多步驟任務。
- 持續型智能體（Persistent Agent）：以目標為導向（Goal-driven），在使用者離線時仍能持續推進工作。

這是老師在課堂上的趨勢整理。產品名稱與時間線屬於簡報中的例子，這裡重點放在能力的差別。

老師提出兩個可能方向：實體 AI（Physical AI），以及把新技術導入專業領域（Domain）。他也強調，方向看得到，不代表能確定何時實現。

對工程學生的提醒：除了學會演算法，也要想「它能幫我解決哪個實際問題？」

2. 支持向量機（Support Vector Machine, SVM）

2.1 它要解決什麼問題？

假設圖上有兩群點，要畫一條線把它們分開。能分開的線可能很多，SVM 要選的是：**讓兩群資料之間的間隔（Margin）盡可能大的決策邊界（Decision Boundary）**。

可以想成在兩群點之間留一條比較寬的走廊，而不是讓分界線緊貼其中一群。

- 二維資料：邊界是一條線。

- 三維資料：邊界是一個平面。
- 更高維資料：統稱超平面（Hyperplane）。

2.2 支持向量（Support Vectors）是哪些點？

在老師的線性可分示意圖中，靠近分隔走廊、卡住間隔的那些點，就是支持向量。它們是決定邊界的重要資料。

課堂問答：刪掉一個離邊界很遠的訓練點，再重新訓練，邊界會怎樣？

在課堂的理想例子中，答案是 **不變**：那個點沒有決定最大間隔。理解重點是「不是每個訓練點對邊界都一樣重要」。這個答案適用於老師示意的非支持向量，不必延伸成任何 SVM 情境都必然不變。

2.3 資料不能用直線分開怎麼辦？

使用核函數（Kernel Function）的想法：在另一個特徵空間（Feature Space）中尋找線性分界，回到原本空間時，就能形成彎曲的邊界。

老師舉了高斯核（Gaussian Kernel），也常稱徑向基底函數核（Radial Basis Function Kernel, RBF Kernel）。

先記住：**原本用直線分不開，不代表換到另一個表示方式後也分不開**。核技巧（Kernel Trick）能避免直接展開所有高維特徵，但不表示運算完全沒有成本。

3. 怎麼比較不同分類模型？

對照逐字稿：約 00:13–00:20。

老師用糖尿病資料集（Diabetes Dataset）快速比較幾個基礎分類器（Base Classifiers）。

模型	白話理解	課堂分類
邏輯斯迴歸（Logistic Regression, LR）	根據特徵估計屬於某一類的機率；名字有「迴歸」，課堂這裡用來做分類。	線性決策邊界
線性判別分析（Linear Discriminant Analysis, LDA）	利用各類資料的分布差異，找出線性的分類方式。	線性模型
k 最近鄰（k-Nearest Neighbors, kNN）	看看附近的 k 個鄰居多半屬於哪一類。	非線性模型
樸素貝氏（Naïve Bayes, NB）	用貝氏定理（Bayes' Theorem）比較各類的可能性；假設給定類別後，各特徵條件獨立。	課堂列在非線性模型組

模型	白話理解	課堂分類
分類與迴歸樹（Classification and Regression Trees, CART）	反覆問「某個特徵是否超過門檻」，走到葉節點（Leaf Node）給答案。	決策樹（Decision Tree）
支持向量機（SVM）	找最大間隔的邊界；使用適當的核函數時，可以處理非線性邊界。	視核函數而定

高斯樸素貝氏（Gaussian Naïve Bayes, GaussianNB）是假設各類中的特徵具有高斯分布（Gaussian Distribution），不是像 SVM 那樣使用高斯核。邏輯斯迴歸也不是靠「假設特徵為高斯分布」來定義。

3.1 交叉驗證（Cross-validation, CV）

不要只看模型在訓練資料上的表現。老師示範 k 折交叉驗證（k-fold Cross-validation），例如分成 10 份：

1. 拿 9 份訓練，留下 1 份驗證。
2. 輪流更換留下的那一份。
3. 整理各輪分數的平均與波動。

分層 k 折（Stratified k-fold）會盡量讓每份資料的類別比例接近原始比例，避免某一折幾乎沒有少數類別。

3.2 不只看平均分數

- 平均（Mean）：通常表現如何？
- 標準差（Standard Deviation）或變異數（Variance）：換一份資料時，分數會不會差很多？

老師用「平均扣掉不確定性（Uncertainty）」提醒大家重視穩健性（Robustness）。先理解成「分數高，也要夠穩」。它不是固定的統計保證；標準差和變異數也不能混為一談。

3.3 課堂示範結果與限制

逐字稿口述的 CV 分數約為：LR 0.769、LDA 0.77、kNN 0.72、NB 0.75、決策樹 0.69、SVM 0.76。這是**當次資料與設定的示範結果**，不是模型的永久排名。

當時 LDA 略好，但各模型還沒仔細調超參數（Hyperparameters），所以只是初步比較。

奧卡姆剃刀（Occam's Razor）的想法：**如果簡單模型在可靠驗證下做得一樣好，就優先考慮簡單模型**。非線性或複雜，不代表一定更好。

4. 集成學習（Ensemble Learning）為什麼有用？

對照逐字稿：約 00:20–00:44。

4.1 多個模型一起決定

先訓練多個基礎模型，再結合它們的預測。

- 分類（Classification）：投票，決定類別。
- 迴歸（Regression）：整合數值預測，例如取平均。

例如三個模型依序預測「有、沒有、有」，多數決（Majority Vote）就判「有」。

4.2 關鍵是多樣性（Diversity）

如果 100 個模型都在同一批資料上犯相同的錯，投 100 票也不會變好。

我們需要的是「錯得不一樣」，不是只把模型數量變多。

多樣性可以來自：改變抽到的資料、資料權重、候選特徵，或切分門檻。

4.3 老師的兩個條件

課堂的二元分類（Binary Classification）理論例子假設：

1. 每個分類器優於隨機猜測（Random Guess）：**失誤率小於 50%，也就是正確率大於 50%**。
2. 分類器的錯誤彼此獨立（Independent）。

實務上很難完全獨立，所以演算法會設法降低錯誤的相關性（Correlation）。這是理論例子成立的條件，不是說所有集成方法都必須完全獨立才有用。

50% 是這裡二元、等機率亂猜的基準；不能直接拿來當多類別或不平衡資料的通用門檻。

4.4 集成失誤率怎麼算？

假設每個分類器都以相同機率 ε 判錯，而且錯誤獨立。三個模型一起投票，只要其中至少兩個判錯，集成就會錯：

$$\varepsilon_{\text{ensemble}} = 3\varepsilon^2(1-\varepsilon) + \varepsilon^3$$

- 恰好兩個錯：有 3 種組合，所以是第一項。

- 三個全部錯：是第二項。

每個失誤率 0.35 時，集成失誤率約 0.282，從 35% 降到約 28%。

對奇數個模型 m，公式是：

$$\varepsilon_{\text{ensemble}} = \sum_{k=(m+1)/2}^{m} \binom{m}{k} \varepsilon^k (1-\varepsilon)^{m-k}$$

不用先被公式嚇到：**把「超過一半模型判錯」的所有情況，其機率加起來**。

老師的 25 個模型例子：每個錯 35%，集成約錯 6%；如果每個反而錯 65%，集成就約錯 94%。模型數量會放大多數決的效果，好壞都會放大。

原筆記修正：不是「把全部模型的失誤率直接相加」，而是加總「多數模型同時判錯」的事件機率。

4.5 投票分類器（Voting Classifier）

可以把不同種類的模型放在一起，例如 LR、SVM、隨機森林。

- 硬投票（Hard Voting）：每個模型投一個類別，採多數決。這是老師本次示範的重點。
- 加權投票（Weighted Voting）：某些模型的票比較重，後面的 AdaBoost 就會用到。

課堂示範的整合分數只比單一模型稍高。老師提醒，在競賽中小幅差距可能影響名次；仍要確認它在可靠驗證資料上有沒有改善。

5. 為什麼 Bagging 常用不穩定的基礎模型？

不穩定分類器（Unstable Base Classifier）：**訓練資料稍微改一點，學到的模型就可能改很多**。

決策樹是課堂代表例子：改幾個點，最先切的特徵或門檻就可能改變，整棵樹也跟著不同。

- 偏差（Bias）：模型的做法太簡化，抓不到真正關係。
- 變異（Variance）：換一組訓練資料，模型結果就大幅變動。

較深的樹通常較有能力貼合資料，但變異可能較高。Bagging 把許多這樣的模型整合，希望讓結果比較穩。

這段主要是在解釋 Bagging 的選模理由。較穩定的模型也可以參與其他集成，例如前面的 Voting Classifier；不要記成「線性模型不能集成」。

理想情況：m 個相互獨立、變異數同為 σ^2 的數值預測，取平均後：

$$\mathrm{Var}(\bar{g}) = \frac{\sigma^2}{m}$$

直覺：每個模型的波動有機會互相抵銷。模型彼此相關時，降低變異的效果會打折扣，這就是後面一直增加多樣性的原因。

6. 自助聚合（Bagging）

對照逐字稿：約 00:50–01:25。

Bagging 的全名是 Bootstrap Aggregating：**先重抽資料，再整合預測**。

6.1 自助抽樣（Bootstrap Sampling）

採用放回抽樣（Sampling with Replacement）：抽完一筆，放回去，再抽下一筆。

例如原始資料是 A、B、C：

- 第一組可能抽到 A、A、C。
- 第二組可能抽到 B、C、C。
- 同一筆可重複出現，也可能完全沒被抽到。

每一組都可以有和原始資料相同的筆數，但包含的「不重複資料」通常比較少。

6.2 Bagging 的流程

1. 從同一份訓練資料，做多次 Bootstrap。
2. 每組抽樣資料各自訓練一個模型。
3. 對新的輸入，各模型分別預測。
4. 用投票或平均整合結果。

各模型的訓練不需要等待前一個模型的答案，所以叫平行集成（Parallel Ensemble）；這不代表一定要使用多台電腦。

6.3 課堂的一維小例子（Toy Example）

原始資料：x = 0.1、0.2、...、1.0；類別 y 依序為：

＋1、＋1、＋1、－1、－1、－1、－1、＋1、＋1、＋1。

決策樹樁（Decision Stump）只做一次切分，例如問「x 是否小於等於門檻 k？」。左右各給一個類別，不能再往下切。

資料的答案呈現「正 → 負 → 正」，單一樹樁只能切一刀，無法一次完整表達；最佳正確率是 **70%**。

老師用不同 Bootstrap 資料訓練多個樹樁，得到不同切點與左右標籤，最後對原始 10 個點做多數決，示範出 100% 的正確率。

這個 100% 是示範資料上的結果，沒有證明對未見資料也能 100%。

標籤是＋1／－1 時，可以加總各模型輸出，再取正負號（Sign）做多數決。偶數個模型可能平手，需另外訂規則。

6.4 為什麼只看到約 63.2% 的不同資料？

原始資料有 n 筆，放回抽 n 次。對某一筆資料來說：

$$P(\text{一次沒抽到}) = 1 - \frac{1}{n}$$

$$P(\text{n 次都沒抽到}) = \left(1 - \frac{1}{n}\right)^n$$

因此至少被抽到一次的機率為：

$$1 - \left(1 - \frac{1}{n}\right)^n \longrightarrow 1 - e^{-1} \approx 0.632$$

抽樣清單仍有 n 筆，只是有重複，所以平均約 63.2% 的原始資料會出現，約 36.8% 沒出現。

這是大樣本的近似與平均，不是每次固定比例。n = 10 時，平均涵蓋率約 65.1%。

6.5 袋外資料（Out-of-bag, OOB）

某棵樹沒抽到的資料，對那棵樹而言是沒有參與訓練的資料，可以拿來評估。

每棵樹的袋外資料不同，不是整片森林共用固定的 36.8%。

對某一筆資料，使用「訓練時沒看過它」的那些樹做預測，再和真實答案比較，得到袋外分數（OOB Score）。

老師提到可設定 `oob_score=True`。這是方便的內建評估方式；若要確認最終泛化能力（Generalization），仍要妥善保留獨立測試資料。

6.6 老師提到的實作參數

- `n_estimators`：基礎模型數量；樹模型時就是幾棵樹。
- `max_samples`：每個模型使用多少抽樣資料。
- `bootstrap`：是否放回抽樣。
- `random_state`：固定隨機性，方便重現結果。
- `n_jobs`：運算使用的平行工作數。

操作主線是擬合（Fit）→ 預測（Predict）→ 評估（Evaluate）。不要只記 API 名稱，要知道它對應哪個步驟。

7. 隨機森林（Random Forest, RF）

對照逐字稿：約 01:25–01:49。

7.1 為什麼 Bagging 還不夠？

假設某個主導特徵（Dominant Feature）特別強，每棵樹即使抽到的資料不同，仍然第一刀就挑它，後面也可能長得很像。

樹長得很像 → 錯誤容易相關 → 集成效果受限。

7.2 在每個節點，只開放一部分特徵

隨機森林除了對資料做 Bootstrap，還在**每個節點切分時，隨機抽一部分候選特徵（Feature Sampling）**，再從其中選最佳切分。

例如有 9 個特徵，這次只抽 3 個來比較。最強特徵可能這次沒抽到，其他特徵就有機會上場。

不是永遠刪掉最強特徵，也不是整棵樹從頭到尾只能用同一組特徵。

老師提到分類時常用候選特徵數約為總特徵數的平方根，例如 64 個特徵，每個節點考慮約 8 個；這是常用設定思路，不是所有情況都必須遵守。

先記成：**隨機森林 = 資料抽樣 + 節點特徵抽樣 + 多棵樹整合**。

它不必讓每棵樹單獨最強，而是兼顧單棵樹的能力與整片森林的多樣性。

7.3 手寫數字辨識示範

老師使用 8 × 8 像素（Pixels）的手寫數字資料：

- 約 1,797 筆樣本。
- 每張圖 64 個像素，因此有 64 個特徵。
- 標籤（Label）是數字 0–9，屬於多類別分類（Multiclass Classification）。

用隨機森林訓練後，課堂示範得到約 98% 的表現。老師強調這是相對乾淨的示範資料，不能拿來保證真實應用的準確率。

8. 特徵重要性與評估報告

8.1 特徵重要性（Feature Importance）

隨機森林可以記錄：某個特徵用來切分時，平均讓資料的不純度（Impurity）降低多少。

直覺：**哪個特徵常常能幫忙把混在一起的類別分開，它對這個模型就比較重要**。

手寫數字例子中，圖中央有筆畫的像素通常較重要；邊緣空白像素通常較不重要。

可透過 `feature_importances_` 查看。這類重要性通常經過正規化（Normalization），所有特徵加總為 1；某特徵不需要接近 1 才算重要。

特徵選擇（Feature Selection）可以參考重要性，但是否刪除特徵，仍要看驗證結果。特徵重要也不等於證明它造成了結果。

8.2 老師的校舍耐震案例

老師介紹一個既有校舍耐震評估的資料案例，候選特徵包括樓地板面積、譜加速度需求（Spectral Acceleration Demand）、鋼筋抗拉強度、Y 向牆量，以及建造年份（Built Year）。

課堂公布的最重要特徵是 **建造年份**。老師將它解釋為可能反映某些年代的材料或施工背景，例如海砂相關問題。

學習重點：資料分析可能揭露專業直覺沒先想到的線索，幫助安排調查或補強優先順序。這是老師案例中的結果與解釋，不能推成「所有同年份建築都一樣」或只靠年份決定是否拆除。

8.3 SHAP 是什麼？

老師補充夏普利加性解釋（SHapley Additive exPlanations, SHAP）：用來解釋各特徵對模型預測的貢獻。逐字稿的「shape／share」應依此理解。

隨機森林內建重要性可先看整體哪些特徵常幫上忙；SHAP 則能進一步分析某個預測中，各特徵把結果往哪個方向推動。本堂只是提到這個工具，沒有完整教學。

8.4 評估報告的英文術語

老師在手寫數字例子中先帶過，下次會再詳細解釋。

指標	用某一類當目標類別時的白話意思
正確率（Accuracy）	全部預測中，有多少比例判對。
精確率（Precision）	模型說是這一類的資料中，有多少真的屬於這一類。
召回率（Recall）	真正屬於這一類的資料中，模型找回多少。
F1 分數（F1 Score）	Precision 與 Recall 的調和平均（Harmonic Mean），兼顧兩者。
樣本數（Support）	評估資料中，真正屬於該類別的樣本數。
宏平均（Macro Average）	先分別計算各類別的指標，再讓每類等權平均。
混淆矩陣（Confusion Matrix）	把真實類別和預測類別交叉列出，看哪些類別容易互相搞混。

混淆矩陣不只看總分，也能告訴你：模型特別容易把哪個數字誤認為哪個數字。

9. 極端隨機樹（Extremely Randomized Trees, Extra Trees, ET）

對照逐字稿：約 01:49–02:01。

隨機森林已經隨機挑候選特徵；Extra Trees 再把**切分門檻（Split Threshold）也隨機化**，增加樹的差異。

例如森林原本會尋找某特徵的最佳門檻，Extra Trees 則先產生隨機候選門檻，再比較候選切分的效果。

它不是完全不看資料亂切，而是在候選切分的產生方式上加入更多隨機性。

方法	主要怎麼增加多樣性？	先記這句
Bagging	放回抽樣，產生不同訓練資料。	每個模型拿到不同的練習題組。

方法	主要怎麼增加多樣性？	先記這句
隨機森林（Random Forest）	資料放回抽樣，加上每個節點隨機挑候選特徵。	題組不同，每次可用的線索也不同。
極端隨機樹（Extra Trees）	隨機挑候選特徵，並隨機產生候選切分門檻。	連切哪個位置，也多加一層隨機性。

實作補充：老師用「在 Random Forest 上再加門檻隨機化」來說明。實際使用 scikit-learn 的 ExtraTreesClassifier 時，預設 `bootstrap=False`，通常用完整訓練集建立每棵樹；是否做 Bootstrap 要另外確認設定。

9.1 老師比較了哪些變因？

- 固定樹的複雜度，改變樹的數量（`n_estimators`）。
- 固定樹的數量，改變最大葉節點數（`max_leaf_nodes`）等複雜度設定。

最大深度（`max_depth`）和最大葉節點數都能限制複雜度，但不是完全相同的限制。

9.2 最重要的是測試表現

訓練誤差（Training Error）下降，可能只是模型更會回答看過的題目。要看保留測試誤差（Holdout Test Error）是否真的改善。

老師示範中，三種方法的訓練曲線不同，但測試表現可能接近。

樹更多或更複雜，不保證每次都更好；樹數增加也常出現改善趨緩。

老師提醒競賽的公開排行榜（Public Leaderboard）與私有排行榜（Private Leaderboard）：一直依公開分數改模型，也可能對那批資料過度擬合（Overfitting）。

10. 從平行集成走向序列集成（Sequential Ensemble）

對照逐字稿：約 02:01–02:09。

比較	平行集成（Parallel Ensemble）	序列集成（Sequential Ensemble）
訓練關係	各基礎模型可各自訓練。	下一輪利用前面預測的不足，決定怎麼訓練。
課堂核心直覺	整合多個結果，讓波動變小。	一步一步補誤差。
常見基礎模型	Bagging 常搭配較深、變異較高的樹。	Boosting 常搭配淺樹等弱學習器。

比較	平行集成（Parallel Ensemble）	序列集成（Sequential Ensemble）
代表方法	Bagging、Random Forest、Extra Trees。	AdaBoost、Gradient Boosting。

弱學習器（Weak Learner）不是「比亂猜還差的模型」，而是能力有限、但仍能提供有用訊息的簡單模型。

老師的課堂問答：**為什麼 Bagging 偏好深樹，AdaBoost 偏好樹樁？**

因為 Bagging 的主要思路是整合後減少變異；Boosting 則逐輪補上簡單模型做不到的部分。這是常見搭配的直覺，不是強制規則。

序列訓練也不是反覆修改同一棵樹就結束，而是逐輪加入新的基礎模型，最後把它們合起來。

Boosting 不一定要做 Bootstrap。它和 Bagging 的關鍵差別，是後一輪依賴前面預測的結果。

11. 自適應提升（Adaptive Boosting, AdaBoost）

對照逐字稿：約 02:09–02:25。

11.1 核心：上次做錯的題目，下次多注意

想像準備考試：做對的題目少花一點時間，做錯的題目多練幾次。

AdaBoost 就是用樣本權重（Sample Weight）做到這件事：

1. 一開始每筆資料權重相同。
2. 訓練第一個簡單分類器。
3. 找出判錯的資料，提高它們下一輪的權重。
4. 依新的權重訓練下一個分類器。
5. 重複多輪，最後用加權投票整合所有分類器。

11.2 先分清兩種權重

- **樣本權重 w：**哪筆訓練資料比較值得注意？每一筆資料各有一個。
- **模型權重 α：**最後投票時，哪個分類器的票比較重？每一輪模型各有一個。

α 會同時影響下一輪樣本權重的更新，以及最後的模型投票；但它和 w 不是同一種權重。

11.3 必要公式：二元 AdaBoost

此處標籤與預測都用＋1／－1，依老師的二元示例整理。

第 t 輪的加權失誤率（Weighted Error Rate）：

$$\varepsilon_t = \frac{\sum_{i=1}^{n} w_i^{(t)} \, \mathbf{1}[h_t(x_i) \neq y_i]}{\sum_{i=1}^{n} w_i^{(t)}}$$

指示函數（Indicator Function）在判錯時等於 1、判對時等於 0。權重已加總為 1 時，分母 就是 1。

不是只數「錯幾筆」，而是看「錯的那些資料，權重合計多少」。

由失誤率決定模型權重：

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$$

當失誤率低於 0.5，α 為正；模型表現較好，最後的票通常也比較重。

更新樣本權重：

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp\left(-\alpha_t y_i h_t(x_i)\right)}{Z_t}$$

- 判對：真實 y 與預測 h 同號，乘積是＋1，乘上小於 1 的因子。
- 判錯：兩者異號，乘積是－1，乘上大於 1 的因子。
- Z 是正規化因子（Normalization Factor），把更新後權重加總調回 1。

最後的加權投票：

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

先讓每個模型的答案乘上它的 α，再加總、取正負號。

11.4 老師的 10 個點例子

第一輪每點權重 0.1，共錯 3 點：

$$\varepsilon_1 = 0.3, \qquad \alpha_1 = \frac{1}{2}\ln\frac{0.7}{0.3} \approx 0.424$$

正規化後：

- 三個判錯點，各約 0.167。
- 七個判對點，各約 0.0714。

所以下一輪三個難題合計約占一半權重，模型會更重視它們。

第二輪即使仍錯 3 點，也不能直接說失誤率仍是 0.3；要看這 3 點現在的權重。

課堂用幾個水平／垂直的樹樁切分，最後加權結合，示範在那些點上全部判對。這是理解機制的例子，仍不等於未見資料的保證。

下一輪會**更有動機**處理高權重錯例，但不保證能全部改對。這和神經網路的注意力機制（Attention）也不是同一個演算法。

12. 本堂最後預告：梯度提升與 XGBoost

對照逐字稿：約 02:25–02:27；下次詳細講。

12.1 梯度提升（Gradient Boosting）

AdaBoost 改變樣本權重；梯度提升讓新模型學習「目前還沒預測好的部分」。

在平方誤差的迴歸例子中，可以先把它理解成學習殘差（Residual）：

$$r_i = y_i - \hat{y}_i$$

例如真值 10，目前預測 7，殘差是 3。下一個模型試著補上這個差距。

更一般地，新模型是朝損失函數（Loss Function）的負梯度（Negative Gradient）方向改進；不同損失下，不一定直接等於「真值減預測值」。

學習率（Learning Rate）控制每一輪補多少：避免一次修得太猛。詳細推導本堂尚未展開。

12.2 極端梯度提升（eXtreme Gradient Boosting, XGBoost）

老師預告會從梯度提升模型（Gradient Boosting Machine, GBM）講到 XGBoost，涉及一階導數、二階資訊／海森矩陣（Hessian）、正則化（Regularization）或懲罰項（Penalty Term），以及超參數調整。

簡報也列出 LightGBM（Light Gradient Boosting Machine）與 CatBoost。這些方法的詳細比較不是本堂已完成的內容；不能把簡報中的性能形容詞當成所有資料集上的固定排名。

---

13. 複習時，先能回答這 8 題

1. **SVM 在找什麼？**最大間隔的決策邊界；支持向量是決定它的重要資料點。
2. **集成為什麼可能比單一模型好？**模型提供有用訊息，而且錯誤不完全相同。
3. **集成失誤率是直接加各模型失誤率嗎？**不是，是計算多數模型同時判錯的機率。
4. **Bagging 改變什麼？**放回抽樣產生不同訓練資料，再整合預測。
5. **63.2% 是什麼？**抽 n 次後，平均有約 63.2% 的不同原始資料至少出現一次。
6. **Random Forest 多做什麼？**每個節點只考慮隨機抽到的一部分候選特徵。
7. **Extra Trees 再多做什麼？**隨機產生候選切分門檻；是否 Bootstrap 看設定。
8. **AdaBoost 怎麼補錯？**提高判錯樣本的權重，逐輪訓練，最後做加權投票。

> 📌 **最值得記住：多樣性（Diversity）、降低變異（Reduce Variance）、逐輪補誤差，以及用未參與訓練的資料驗證。**
> 模型數量、複雜度或訓練分數，都不能代替對泛化能力的檢查。

課堂來源與查核補充

- 🖼 第五週簡報
- 主簡報：Classification and Regression II
- 課堂逐字稿：使用你提供的約 2 小時 28 分鐘 PDF，已略去休息時間私人對話、設備操作與重複語句。
- Bagging 補充講義（保留你原本的連結）
- scikit-learn 官方集成方法說明：查核 Extra Trees 的門檻隨機化與抽樣設定。
- scikit-learn 官方樸素貝氏說明：查核 GaussianNB 的高斯分布假設。
- SHAP 官方說明：查核正式名稱與解釋用途。